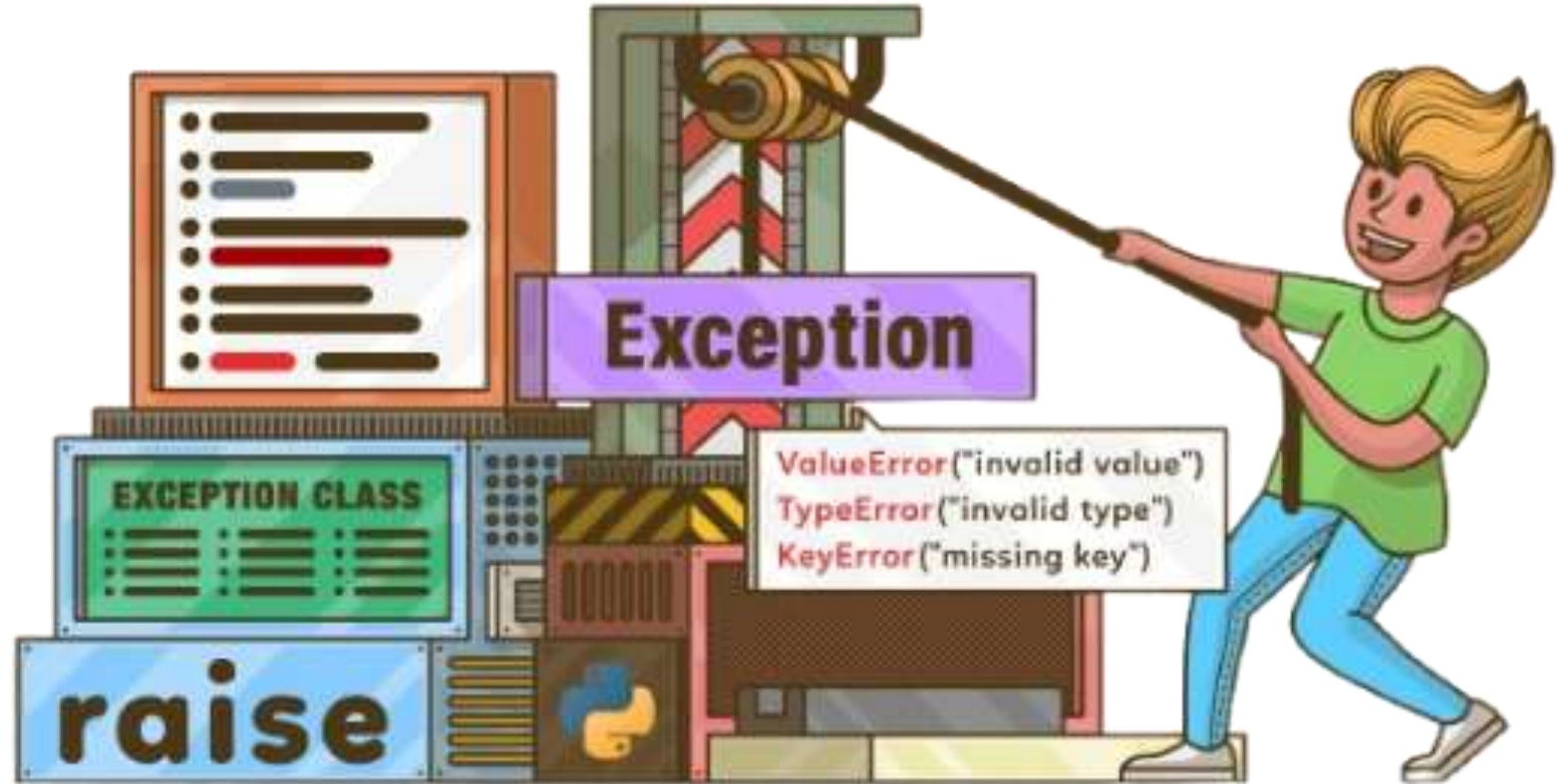




*Exceptions in
Python*

What is an Exception?

An **Exception** is an event that occurs during the execution of a program and disrupts the normal flow of instructions.

- Python raises an exception when it encounters an error.
- If not handled, the program crashes.

Real-World Analogy:

Think of an exception like a "**red light**" while driving — it interrupts your journey and forces you to take action.

☐ Example:

```
print(10 / 0)    # ZeroDivisionError
```



Types of Errors in Python

□ 1. Syntax Errors

- Errors in the structure of code.
- Found before code execution.

```
# Missing colon  
if True  
    print("Hello")
```

2. Runtime Exceptions

- Occur while the program is running.
- Handled using try-except.

```
print(5 + "5") # TypeError
```



Common Exception Types

! Exception	 Meaning	□ Example
ZeroDivisionError	Divide by zero error	10 / 0
TypeError	Invalid operation between data types	"5" + 5
ValueError	Wrong type of value	int("abc")
IndexError	Invalid index in a list	lst[10] when list has 5 elements
KeyError	Accessing missing dict key	mydict["abc"]



➤ Why Handle Exceptions?

- Prevent program from **crashing**.
- Show user-friendly error messages.
- Help in **debugging**.
- Essential for **robust** applications.

🔗 Example without exception handling:

```
num = int(input("Enter number: ")) # crashes on "abc"  
print(100 / num)
```

🔗 Example with exception handling:

```
try:  
    num = int(input("Enter number: "))  
    print(100 / num)  
except (ValueError, ZeroDivisionError):  
    print("⚠ Please enter a valid, non-zero number.")
```



➤ Basic Try-Except Syntax

```
try:  
    # risky code  
except ErrorType:  
    # handling code
```

🔗 Example:

```
try:  
    a = int("xyz")  
except ValueError:  
    print("Invalid input!")
```

- ❑ You can catch **multiple exceptions** using separate blocks or tuples.



➤ try-except-else-finally Flow

```
try:
    # try block
except:
    # runs if error occurs
else:
    # runs if no error occurs
finally:
    # always runs
```

🎯 Example:

```
✓ try:
    num = int(input("Enter number: "))
    result = 10 / num
✓ except ZeroDivisionError:
    print("Cannot divide by zero!")
✓ else:
    print(f"Result: {result}")
✓ finally:
    print("End of program ✅")
```



Raising Exceptions Manually

You can use `raise` to throw an exception **deliberately**.

```
age = int(input("Enter age: "))  
if age < 0:  
    raise ValueError("Age cannot be negative!")
```

🔗 Use in **form validation**, **input checking**, or **logic enforcement**.



Custom Exception Example

Create your own error class using class.

```
class MyError(Exception):  
    pass  
  
try:  
    raise MyError("This is a custom error")  
except MyError as e:  
    print("Caught:", e)
```



➤ Multiple Exceptions

```
try:
    x = int(input("Enter number: "))
    y = 10 / x
except ValueError:
    print("⚠ Invalid number entered.")
except ZeroDivisionError:
    print("⚠ Cannot divide by zero.")
```



Best Practices

- Catch **specific** exceptions (not general except)
- Use finally for cleanup code
- Use else only if no error occurs
- Don't **hide** exceptions unintentionally



